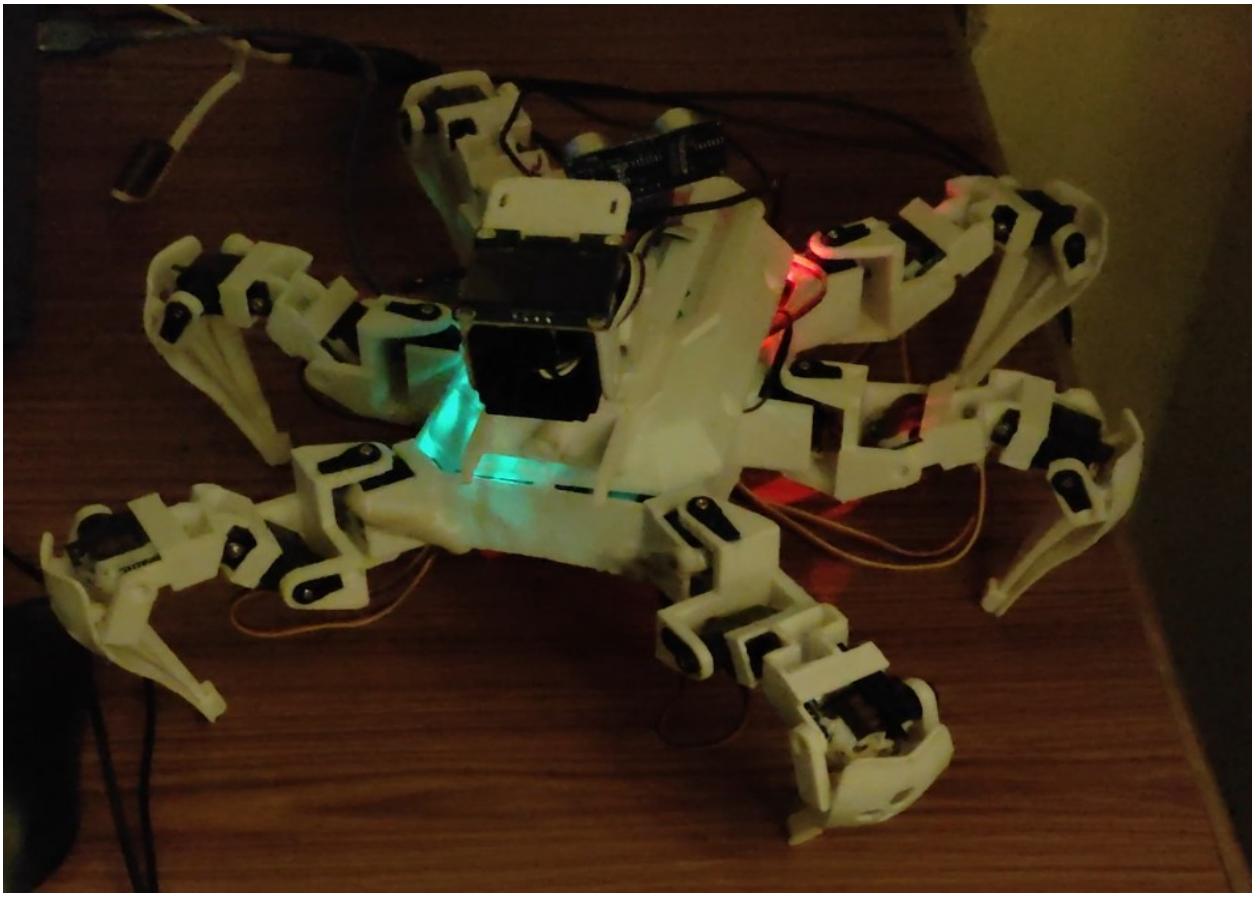
🦷 Hexapod Robot — Wireless Controlled with Sensor Dashboard



A 6-legged walking robot with wireless joystick control, ESP-NOW communication, and a live environmental sensor dashboard.

📖 Table of Contents

- Overview
- System Architecture
- Hardware Components
- Wiring & Connections
- How It Works
- Firmware — Arduino Nano
- NodeMCU — Sensor Hub & Serial Bridge
- Wireless Controller
- Web Dashboard
- Getting Started
- Folder Structure
- Future Improvements

🏗️ System Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                    HEXAPOD ROBOT BODY                        │
│                                                              │
│   ┌──────────────────┐   RX/TX   ┌──────────────────────┐   │
│   │ Arduino Nano     │◄──────────►│ NodeMCU (ESP8266)    │   │
│   │ (Firmware)       │  Serial    │ - DHT11 Sensor       │   │
│   │ Servo PWM        │            │ - ESP-NOW Receiver   │   │
│   └──────────────────┘            │ - Web Server         │   │
│         │                         └──────────┬───────────┘   │
│   18x Servos                                 │ WiFi          │
└──────────────────────────────────────────────┼───────────────┘
                                               │
                                               ┌──────────┐
                                               │ ESP-NOW  │
                                               └──────────┘
                                               ▼          ▼
                                   ┌──────────────────┐  ┌──────────────────┐
                                   │ Custom Controller │  │ Web Dashboard    │
                                   │ - 2x Joystick     │  │ (Browser / Phone)│
                                   │ - 8x Push Buttons │  │ - Temp & Humidity│
                                   │ - ESP8266 (TX)    │  │ - Live Updates   │
                                   └──────────────────┘  └──────────────────┘
```

🔩 Hardware Components

Robot Body

Component	Quantity	Purpose
Arduino Nano	1	Main motion controller
NodeMCU ESP8266	1	WiFi, ESP-NOW bridge, sensor hub
DHT11 Sensor	1	Temperature & humidity readings
Servo Motors (SG90 / MG996R)	18	3 per leg × 6 legs
LiPo / 18650 Battery Pack	1	Power supply
PCA9685 Servo Driver (optional)	1–2	PWM expansion for servos

Component	Quantity	Purpose
Hexapod Frame	1	3D printed

Wireless Controller

Component	Quantity	Purpose
Esp32 devkit v1	1	ESP-NOW transmitter
Joystick Module (KY-023)	2	Movement & direction control
Push Buttons	8	Gait modes, speed, special moves
3.7V LiPo Battery	1	Portable power
Custom PCB / Protoboard	1	Mounting all components

🔌 Wiring & Connections

Arduino Nano ↔ NodeMCU Serial Bridge

```
Arduino Nano        NodeMCU ESP8266
     RX ──────────────────── TX (D1 / GPIO1)
     TX ──────────────────── RX (D2 / GPIO3)
    GND ──────────────────── GND
```

> ⚠️ Note: The Arduino Nano operates at 5V logic while the NodeMCU uses 3.3V.
> So only connect tx of node mcu to the nano's rx.

DHT11 → NodeMCU

```
DHT11 Pin    NodeMCU Pin
   VCC ──────── 3.3V
   GND ──────── GND
  DATA ──────── D4 (GPIO2)
```

Controller Joystick Wiring

```
Joystick Module    NodeMCU (Controller)
```

```
VCC ──────────── 3.3V
GND ──────────── GND
VRX ──────────── A0 (analog)
VRY ──────────── D5
 SW ──────────── D6
```

(Repeat for second joystick on different pins)

Push Buttons (Controller)

All 8 buttons wired between a digital GPIO pin and GND. Internal pull-ups enabled in firmware.

```
Button 1 → D0   Button 5 → D7
Button 2 → D1   Button 6 → D8
Button 3 → D2   Button 7 → TX
Button 4 → D3   Button 8 → RX
```

⚙️ How It Works

1. Wireless Controller → Esp32 (Robot)

The handheld controller reads joystick axes and button states, encodes them into a compact struct, and broadcasts via ESP-NOW to the robot's NodeMCU MAC address. ESP-NOW is a connectionless WiFi protocol that provides low-latency (~1ms) peer-to-peer communication without needing a router.

```
[Controller ESP8266] ──ESP-NOW──► [Robot NodeMCU]
      Joystick + Button data          Receive & decode
```

2. NodeMCU → Arduino Nano (Serial Commands)

On receiving an ESP-NOW packet, the NodeMCU decodes the joystick/button data and sends a simple single-byte or short ASCII command over Serial (TX→RX) to the Arduino Nano.

Example command map:

```
'F'  → Walk Forward
'B'  → Walk Backward
'L'  → Turn Left
'R'  → Turn Right
'S'  → Stop
'1'  → Gait Mode 1 (tripod)
'2'  → Gait Mode 2 (wave)
'W'  → Wave gesture
```

3. Arduino Nano — Motion Execution

The Nano listens on its Serial port. On receiving a command character, it executes the corresponding servo sequence to produce the desired movement. All servo timing and gait logic lives here.

4. DHT11 Sensor — Live Dashboard

Simultaneously, the NodeMCU reads the DHT11 every few seconds and serves the data as JSON via a simple HTTP server. The web dashboard polls or uses WebSockets to display live temperature and humidity.

💻 Firmware — Arduino Nano

File: `arduino_nano/hexapod_firmware/hexapod_firmware.ino`

The firmware initializes all 18 servos using the `Servo.h` library (or PCA9685 via `Adafruit_PWMServoDriver.h`) and maps each to a leg/joint index. The main loop reads from `Serial` and dispatches movement functions.

```cpp
```

```
void loop() {
  if (Serial.available()) {
    char cmd = Serial.read();
    switch (cmd) {
      case 'F': walkForward(); break;
      case 'B': walkBackward(); break;
      case 'L': turnLeft(); break;
      case 'R': turnRight(); break;
      case 'S': standStill(); break;
      // ... more commands
    }
  }
}
```

Key parameters you can tune:

Parameter	Default	Description
`STEP_HEIGHT`	30	How high the leg lifts (degrees)
`STEP_SPEED`	15 ms	Delay between servo increments
`STRIDE_LENGTH`	20	How far each leg swings forward
`BAUD_RATE`	9600	Must match NodeMCU Serial config

📡 NodeMCU — Sensor Hub & Serial Bridge

File: `nodemcu_robot/nodemcu_robot.ino`

The NodeMCU runs three tasks concurrently:

1. ESP-NOW Receiver — registers a callback that fires on each received packet
2. Serial Bridge — forwards decoded commands to Arduino Nano via `Serial.write()`
3. Web Server — serves sensor data as JSON on `http://<robot-ip>/data`

```cpp
```

```
// ESP-NOW receive callback
void onDataReceived(uint8_t *mac, uint8_t *data, uint8_t len) {
  ControlPacket pkt;
    memcpy(&pkt, data, sizeof(pkt));
    char cmd = decodeCommand(pkt);
  Serial.write(cmd);              // Forward to Arduino Nano
}

// Web server endpoint
server.on("/data", []() {
    float temp = dht.readTemperature();
    float hum  = dht.readHumidity();
  String json = "{\"temp\":" + String(temp) + ",\"humidity\":" + String(hum) + "}";
  server.send(200, "application/json", json);
});
```

🕹️ Wireless Controller

File: `controller/controller.ino`

The controller ESP8266 reads both joysticks and all 8 buttons, packs values into a struct, and sends it over ESP-NOW every ~50ms.

```cpp
typedef struct {
    int  joy1_x;
    int  joy1_y;
    int  joy2_x;
    int  joy2_y;
  byte buttons;             // bitmask for 8 buttons
} ControlPacket;
```

Button Functions (default mapping):

Button	Function
BTN 1	Tripod Gait
BTN 2	Wave Gait
BTN 3	Speed Up
BTN 4	Speed Down
BTN 5	Wave Hello
BTN 6	Dance Mode
BTN 7	Sit Down
BTN 8	Stand Up

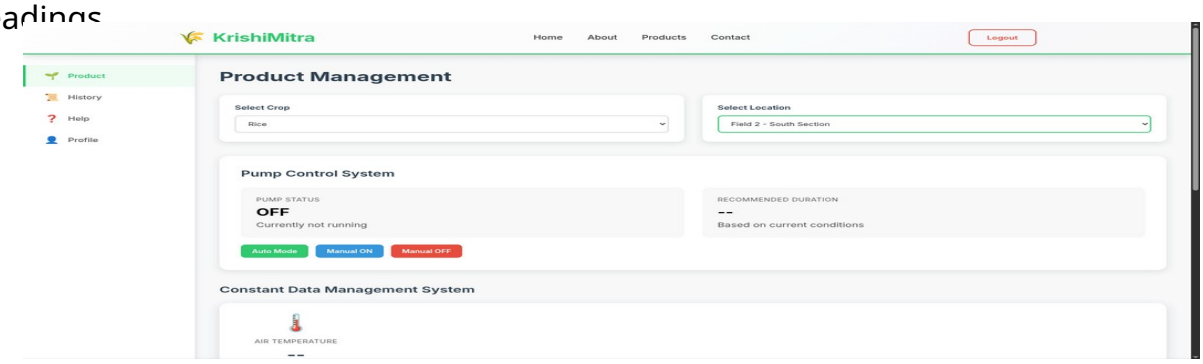
To pair the controller with your robot, set the robot NodeMCU's MAC address in `controller.ino`:

```cpp
uint8_t robotMAC[] = {0xXX, 0xXX, 0xXX, 0xXX, 0xXX, 0xXX};
```

🌐 Web Dashboard

File: `dashboard/index.html`

A standalone HTML page that connects to the robot NodeMCU's IP address and displays live DHT11 readings



Features:

- Live temperature (°C / °F toggle)
- Live humidity percentage
- Auto-refresh every 3 seconds

- Works on any browser on the same WiFi network

Setup:

1. Connect your phone/PC to the same WiFi network as the NodeMCU
2. Find the robot's IP address (printed to Serial on boot, or check your router)
3. Open `dashboard/index.html` in a browser and enter the IP
4. Or navigate directly to `http://<robot-ip>/data` to see raw JSON

🚀 Getting Started

Prerequisites

- Arduino IDE 1.8+ or Arduino IDE 2.x
- ESP8266 board package installed (`http://arduino.esp8266.com/stable/package_esp8266com_index.json`)
- Libraries:
  - `Servo.h` (built-in)
  - `DHT sensor library` by Adafruit
  - `ESP8266WiFi.h` (included with ESP8266 package)
  - `espnow.h` (included with ESP8266 package)
  - `ESP8266WebServer.h`

Upload Steps

Step 1 — Flash Arduino Nano

```
1. Open  arduino_nano/hexapod_firmware/hexapod_firmware.ino
2. Select board: Arduino Nano
3. Select correct COM port
4. Upload
```

Step 2 — Get Robot NodeMCU MAC Address

```
1. Flash  nodemcu_robot/get_mac/get_mac.ino  temporarily
2. Open Serial Monitor at 115200 baud
3. Note the printed MAC address
4. Put it in controller/controller.ino
```

Step 3 — Flash Robot NodeMCU

```
1. Edit nodemcu_robot/nodemcu_robot.ino
2. Set your WiFi SSID and password
3. Select board: NodeMCU 1.0 (ESP-12E Module)
4. Upload
```

Step 4 — Flash Controller NodeMCU

```
1. Edit controller/controller.ino
2. Paste robot MAC address
3. Select board: NodeMCU 1.0
4. Upload
```

Step 5 — Power On & Test

```
1. Power on the robot
2. Open Serial Monitor on robot NodeMCU → note IP address
3. Open dashboard/index.html → enter IP
4. Pick up controller → move joystick → hexapod should move!
```

📁 Folder Structure

```
hexapod/
├── arduino_nano/
│   ├── hexapod_firmware/
│   │   └── hexapod_firmware.ino     # Main motion firmware
│   └── get_mac/
│       └── get_mac.ino              # Utility: print MAC address
│
├── nodemcu_robot/
│   └── nodemcu_robot.ino            # ESP-NOW receiver + web server + DHT11
│
├── controller/
```

```
│   └── controller.ino          # ESP-NOW transmitter + joystick + buttons
│
├── dashboard/
│   └── index.html              # Web dashboard for sensor data
│
├── images/
│   ├── hexapod_main.jpg
│   ├── dashboard.png
│   ├── wiring_diagram.png
│   └── controller.jpg
│
└── README.md
```

🔮 Future Improvements

- [ ] Add more sensors: MPU-6050 (IMU / balance), ultrasonic (obstacle avoidance)
- [ ] OTA (Over-the-Air) firmware updates via NodeMCU
- [ ] MQTT integration for Home Assistant / Node-RED dashboard
- [ ] Replace DHT11 with DHT22 for better accuracy
- [ ] Add battery voltage monitoring to dashboard
- [ ] Inverse kinematics for smoother, terrain-adaptive gait
- [ ] Mobile app controller (Flutter / React Native)
- [ ] Camera module on NodeMCU for FPV streaming

📜 License

This project is open-source under the MIT License.

🙏 Acknowledgements

- Arduino Servo Library
- Adafruit DHT Library
- ESP8266 Arduino Core
- Hexapod gait algorithms inspired by various open-source walking robot projects

---

Built with ❤️— Happy hacking!